

COPY_THIS_TO_CODEX.md

=====

Paste this into Codex

You are working in a Next.js App Router + TypeScript + Supabase internal operations app for a furniture retail business called ****ChaChaCha Ops****.

Goal

Turn the existing scaffold into a proper production-minded MVP app. Do not treat the current repo as finished. It is a shell with pages, RBAC helpers, validators, and Supabase utilities. Your job is to make the app real by adding database migrations, Supabase workflows/RPCs, real data reads/writes, RLS, and tests.

Current stack

- Next.js App Router
- TypeScript
- Supabase Auth + Postgres
- Tailwind CSS
- shadcn-style local UI components
- Zod validation

Critical business rules

1. Inventory movements are the source of truth.
2. Inventory balances are read/materialized state derived from movements.
3. Inventory states must support 'on_hand', 'reserved', and 'sold'.
4. No stock reservation is allowed without payment/deposit.
5. Deposit creates reservation.
6. Delivery completion converts 'reserved' to 'sold'.
7. Sales can be 'cash', 'credit', or 'mixed'.
8. Multiple inventory locations must be supported.
9. Credit accounts need installment tracking.
10. Collections workflow must track actions, promises, and payments.
11. Delivery workflow must track status and completion proof.
12. Daily cash reconciliation must compare expected vs counted cash.
13. Owner dashboard must show daily operational control metrics.
14. Sensitive operations must be guarded by RBAC, server-side checks, Supabase RLS, and audit logs.

Existing app structure to preserve

- 'app/(auth)/login' for login.
- 'app/(dashboard)/owner' for owner dashboard.
- 'app/(dashboard)/sales' for sales entry.
- 'app/(dashboard)/inventory' for inventory movements.
- 'app/(dashboard)/credit' for credit application review.
- 'app/(dashboard)/collections' scaffold.
- 'app/(dashboard)/delivery' scaffold.
- 'app/(dashboard)/finance' scaffold.
- 'lib/permissions/*' for RBAC permission map.
- 'lib/auth/*' for server-side permission checks.
- 'lib/validators/*' for Zod schemas.
- 'lib/supabase/*' for Supabase clients.
- 'components/ui/*' for reusable UI primitives.

First implementation task

Start by creating a real Supabase database layer:

1. Add 'db/migrations/0001_core_schema.sql' with tables:
 - 'users'
 - 'roles'
 - 'user_roles'
 - 'locations'
 - 'customers'
 - 'suppliers'
 - 'products'
 - 'product_variants'
 - 'price_lists'
 - 'price_list_items'
 - 'inventory_balances'
 - 'inventory_movements'
 - 'sales'
 - 'sale_lines'
 - 'payments'
 - 'credit_applications'
 - 'credit_accounts'
 - 'installment_schedule'
 - 'collection_actions'
 - 'deliveries'
 - 'expenses'
 - 'cash_sessions'
 - 'audit_logs'
2. Add enums/check constraints for statuses and movement types.
3. Add 'db/migrations/0002_seed_roles.sql' for the roles used by 'lib/permissions/permissions.ts'.
4. Add 'db/migrations/0003_inventory_rpc.sql' for:
 - 'fn_confirm_reservation(sale_id uuid)'
 - 'fn_complete_delivery(delivery_id uuid)'
 - 'fn_post_inventory_movement(...)'
5. Add 'db/migrations/0004_rls_policies.sql' for role-aware access control.
6. Add 'db/migrations/0005_audit_triggers.sql' for audit logs.

Then wire app behavior

1. Replace hardcoded dashboard/sales/inventory/credit sample arrays with Supabase queries.
2. Update sales server action so it creates:
 - customer if needed,
 - sale,
 - sale lines,
 - payment/deposit,
 - reservation via 'fn_confirm_reservation'.
3. Update delivery route/action so completion calls 'fn_complete_delivery'.
4. Update credit review so approval creates credit account and installment schedule.
5. Update collections so payments apply to oldest unpaid installments.
6. Update finance close so it calculates expected cash and variance.

Rules for implementation

- Do not let clients directly update inventory balances.
- Do not manually set sold/reserved quantities from the UI.
- Do not bypass permissions in server actions or route handlers.

- Prefer Supabase RPC for multi-table transactions.
- Keep UI components reusable.
- Keep domain logic out of page components where possible.
- Use Zod schemas for every mutation payload.
- Add tests for permission logic and business workflows.

Expected output from Codex

- A committed branch with migrations, working server actions/routes, and real Supabase-backed pages.
- Updated README setup instructions.
- Tests/checks run and listed clearly.

PROJECT_BRIEF.md

=====

Project Brief: ChaChaCha Ops

Business

ChaChaCha Ops is an internal furniture retail operations app for managing store and warehouse operations.

MVP modules

1. Auth and role-based access control.
2. Owner dashboard.
3. Sales entry.
4. Inventory movements and balances.
5. Credit application review.
6. Credit accounts and installment tracking.
7. Collections workflow.
8. Delivery workflow.
9. Daily cash/finance close.

Non-negotiable control rules

Rule	Why it matters
Inventory movements are source of truth	Prevents manual stock manipulation and reporting drift.
Balances are derived from movements	Makes stock numbers explainable and auditable.
No reservation without payment	Prevents unpaid stock holds.
Deposit creates reservation	Matches retail operating process.
Delivery completion converts reserved to sold	Keeps physical fulfillment tied to stock state.
Multi-location inventory	Supports warehouse/store split.
RBAC everywhere	Internal users should only see/do their job.
Audit logs for critical changes	Owner needs traceability for money, credit, and stock.

MVP user roles

- 'owner'
- 'ops_manager'
- 'salesperson'
- 'cashier'
- 'inventory_controller'

- 'credit_officer'
- 'collections_officer'
- 'delivery_coordinator'

MVP success definition

The MVP is successful when a real user can:

1. Log in.
2. See only allowed modules.
3. Create a sale.
4. Take a deposit/payment.
5. Reserve stock through a movement ledger.
6. Complete delivery and convert reserved stock to sold.
7. Approve credit and generate installment schedules.
8. Log collection actions and payments.
9. Close daily cash.
10. See owner dashboard control metrics.

IMPLEMENTATION_PLAN.md

=====

Implementation Plan

Phase 1: Make the scaffold runnable

- Run 'npm install'.
- Copy '.env.example' to '.env.local'.
- Add Supabase URL and anon key.
- Run 'npm run dev'.
- Confirm '/login', '/owner', '/sales', '/inventory', and '/credit' render.

Phase 2: Create real Supabase schema

Add migrations in this order:

1. '0001_core_schema.sql'
2. '0002_seed_roles.sql'
3. '0003_inventory_rpc.sql'
4. '0004_rls_policies.sql'
5. '0005_audit_triggers.sql'

Phase 3: Auth and RBAC

- Create Supabase Auth user for owner.
- Insert matching app 'users' row.
- Seed 'roles'.
- Insert owner 'user_roles' row.
- Confirm '/owner' works for owner.
- Confirm '/forbidden' works for unauthorized roles.

Phase 4: Real sales and reservation

- Replace sales form hardcoded product/location with real select/search.
- Server action creates 'sales', 'sale_lines', and 'payments'.
- Deposit/full payment calls 'fn_confirm_reservation'.
- 'fn_confirm_reservation' posts 'reserve_out' movements.

Phase 5: Real inventory

- Inventory page reads 'inventory_movements' and 'inventory_balances'.
- Manual adjustments require 'inventory.movement.create'.
- Do not expose direct editing of 'inventory_balances'.

Phase 6: Delivery workflow

- Create delivery records for reserved sales.
- Delivery completion calls 'fn_complete_delivery'.
- Completion posts 'sold_out_delivery' movement rows.

Phase 7: Credit and collections

- Credit approval creates 'credit_accounts' and 'installment_schedule'.
- Collection payments apply to oldest unpaid installments.
- Collection actions log call/visit/promise/payment follow-up.

Phase 8: Finance close and owner dashboard

- Add 'cash_sessions' table.
- Expected cash = cash payments + cash collections for location/date.
- Variance = counted cash - expected cash.
- Owner dashboard reads from SQL views/RPCs instead of raw UI aggregation.

Phase 9: Production hardening

- Add RLS policies.
- Add audit triggers.
- Add tests.
- Generate Supabase TypeScript types.
- Add CI checks for 'npm run typecheck' and 'npm run build'.

DATABASE_BLUEPRINT.md

=====

Database Blueprint

Required tables

Table	Purpose
---	---
'users'	App profile linked to Supabase Auth.
'roles'	RBAC role catalog.
'user_roles'	User-to-role assignment.
'locations'	Store/warehouse inventory locations.
'customers'	Customer profile.
'suppliers'	Supplier/vendor profile.
'products'	Product family/catalog item.
'product_variants'	Sellable SKU/variant.
'price_lists'	Pricing groups/versions.
'price_list_items'	SKU prices per price list.
'inventory_movements'	Source-of-truth stock ledger.
'inventory_balances'	Derived read model by variant/location.
'sales'	Sale header.
'sale_lines'	Sale line items.

```
| 'payments' | Deposits, final payments, credit payments. |
| 'credit_applications' | Credit review request. |
| 'credit_accounts' | Approved account/loan. |
| 'installment_schedule' | Due dates and balances. |
| 'collection_actions' | Calls, visits, promises, collection notes. |
| 'deliveries' | Delivery scheduling and completion. |
| 'expenses' | Store/operations expenses. |
| 'cash_sessions' | Daily cash open/close reconciliation. |
| 'audit_logs' | Immutable operational audit trail. |
```

Required enums/statuses

```
- 'sale_status': 'draft', 'reserved', 'delivery_pending', 'delivered', 'cancelled'
- 'payment_mode': 'cash', 'credit', 'mixed'
- 'payment_method': 'cash', 'card', 'bank_transfer', 'mobile_money'
- 'payment_kind': 'deposit', 'final', 'credit_installment', 'refund'
- 'inventory_movement_type': 'stock_in_purchase', 'stock_in_adjustment', 'reserve_out',
'reserve_release', 'sold_out_delivery', 'transfer_out', 'transfer_in', 'damage_out'
- 'credit_application_status': 'draft', 'submitted', 'under_review', 'approved', 'rejected',
'cancelled'
- 'credit_account_status': 'active', 'closed', 'defaulted', 'restructured'
- 'installment_status': 'pending', 'partial', 'paid', 'overdue'
- 'delivery_status': 'pending', 'scheduled', 'out_for_delivery', 'completed', 'failed',
'cancelled'
- 'cash_session_status': 'open', 'closed'
```

Critical RPC functions

```
### 'fn_confirm_reservation(p_sale_id uuid)'
```

Must:

1. Check payment/deposit exists.
2. Check line stock is available at sale location.
3. Insert 'reserve_out' inventory movements.
4. Update sale status to 'reserved'.
5. Audit the transition.

```
### 'fn_complete_delivery(p_delivery_id uuid)'
```

Must:

1. Check delivery exists and sale is reserved/delivery pending.
2. Insert 'sold_out_delivery' movement rows.
3. Mark delivery 'completed'.
4. Mark sale 'delivered'.
5. Audit the transition.

```
### 'fn_apply_credit_payment(p_credit_account_id uuid, p_amount numeric)'
```

Must:

1. Insert payment row.
2. Apply amount to oldest unpaid installments.
3. Update installment statuses.
4. Audit payment application.

```
### 'fn_close_cash_session(p_location_id uuid, p_session_date date, p_counted_cash numeric)'
```

Must:

1. Calculate expected cash from cash payments/collections.
2. Store counted cash.
3. Store variance.
4. Prevent duplicate close.
5. Audit close.

RLS requirements

- Enable RLS on all operational tables.
- Authenticated users can read their allowed module data.
- Only owners/ops managers can see cross-module sensitive data.
- Inventory balances should be readable but not directly editable by normal users.
- Inventory movements should only be inserted by allowed roles or secure RPCs.
- Financial writes should require cashier/owner/ops manager permissions.

APP_STRUCTURE.md

=====

App Structure Guide

'app/'

Next.js App Router routes.

- '(auth)/login': login screen and login server action.
- '(dashboard)/layout.tsx': shared dashboard shell and navigation.
- '(dashboard)/owner': owner KPI/control dashboard.
- '(dashboard)/sales': sales entry.
- '(dashboard)/inventory': movement ledger UI.
- '(dashboard)/credit': credit application review.
- '(dashboard)/collections': collections scaffold.
- '(dashboard)/delivery': delivery scaffold.
- '(dashboard)/finance': cash close scaffold.
- 'api/**': guarded route-handler boundaries for mutations.

'components/'

Reusable UI pieces.

- 'ui': local shadcn-style primitives.
- 'tables': table helpers and status badges.
- 'dashboards': KPI/dashboard components.

'lib/'

Application logic.

- 'auth': current role lookup and permission guard.
- 'permissions': canonical RBAC map.
- 'supabase': browser/server/middleware clients.
- 'validators': Zod schemas for mutations.
- 'domain': pure business rules.
- 'utils': shared helpers.

'types/'

Generated/shared TypeScript types. Replace placeholder database types with generated Supabase types after schema is created.

'docs/'

Human-readable architecture, workflows, and RBAC notes.

'db/'

Add real Supabase migrations here. This folder is the next major work item.

ACCEPTANCE_CHECKLIST.md

=====

Acceptance Checklist

Setup

- [] 'npm install' succeeds.
- [] '.env.local' is configured.
- [] 'npm run dev' starts app.
- [] 'npm run typecheck' passes.
- [] 'npm run build' passes.

Auth/RBAC

- [] Owner can access all modules.
- [] Salesperson can access sales only.
- [] Cashier can capture payments and close cash.
- [] Inventory controller can post allowed movements only.
- [] Credit officer can review credit applications.
- [] Collections officer can log collection actions/payments.
- [] Delivery coordinator can schedule/complete deliveries.
- [] Unauthorized users see '/forbidden'.

Inventory

- [] Stock changes only through 'inventory_movements'.
- [] 'inventory_balances' is derived and cannot be manually edited from UI.
- [] Reservation posts 'reserve_out'.
- [] Delivery completion posts 'sold_out_delivery'.
- [] Negative stock states are blocked.
- [] Multi-location stock works.

Sales

- [] Sale can be cash, credit, or mixed.
- [] Deposit creates reservation.
- [] No reservation without payment.
- [] Sale lines use real products/variants/prices.

Credit/Collections

- [] Credit application can be approved/rejected.

- [] Approved credit creates account and installments.
- [] Payments apply to installments.
- [] Collection actions are logged.

Delivery

- [] Delivery can be scheduled.
- [] Delivery completion requires allowed role.
- [] Completion converts reserved stock to sold.

Finance close

- [] Cash sessions open/close.
- [] Expected cash is system-calculated.
- [] Variance is stored and visible to owner.

Audit

- [] Sales status changes are audited.
- [] Payments are audited.
- [] Inventory movements are audited.
- [] Credit decisions are audited.
- [] Delivery completion is audited.
- [] Cash close is audited.